

The whole loop is built and tested. None of it has met the real world.

Every phase of the twelve-week plan has been implemented and covered by tests. **What has not happened is the part that decides whether any of it is right:** it has never run on a phone, never talked to a real backend, and never been seen by a contractor.

619 TESTS PASSING	82 SOURCE FILES	14.2k LINES OF CODE	6.9k LINES OF TESTS	19 SECURITY ASSERTIONS
-----------------------------	---------------------------	-------------------------------	-------------------------------	-------------------------------------

Typecheck clean with no unused locals · bundles for Android · 9 migrations apply to a real Postgres · 16 routes resolve.

What is built

Phase 0 — Foundations DONE Expo SDK 57 scaffold, full Postgres schema behind RLS, and the measurement engine: three numbers in, every derived quantity out.	Phase 1 — Capture DONE Jobs, rooms, openings, photos, damage sheet, voice notes. Offline-first throughout, with three queues draining in dependency order.
Phase 2 — Scope and price DONE CSV price import, deterministic auto-scoping from damage, line item editing, and the full estimate calculator with immutable versions.	Phase 3 — Output DONE Estimate PDF, photo report, CSV export, shareable link, send flow, and the job status lifecycle through to closed.
Phase 4 — AI layer DONE All five features, an Edge Function with a per-job spend ceiling, and acceptance tracking wired to the plan's 60% cut line.	Phase 5 — Beta prep PARTIAL Trial and entitlement model, onboarding, EAS build config, permission strings, Sentry. Stripe itself is not built.

The three things that have never happened

There is no sign-in. Nothing has ever synced.

This is the largest gap, and it is load-bearing. Every RLS policy resolves the tenant through `auth.uid()`, and the app never authenticates — the company id is the literal string `'co-1'`, hardcoded in one file.

The consequence: **the app works completely offline and has never written a single row to a server.** Sync would be refused by RLS on every push, and the AI Edge Function rejects any call without a bearer token. The plan scheduled this for Week 1 Day 2 and it was never built.

It has never run on a phone.

Verification so far is a test suite, a typechecker, a Metro bundle and a Postgres container. Every screen, every gesture, every permission prompt and the entire camera and speech path is **unproven on real hardware**. Expo Go will not do — speech recognition and Sentry are native modules, so a development build is required.

The AI has never seen a real response.

Every AI test runs against fabricated JSON. The contract, the resolver and the cost model are solid, but whether the prompts produce *useful* findings on an actual photograph of a wet wall is **entirely unknown**. The acceptance-rate metric exists precisely to answer this, and has no data in it.

What is not built

ITEM	STATE	WHAT IT NEEDS
Authentication Email magic link, company bootstrap	missing	A sign-in screen and a call to the <code>bootstrap_company</code> function that already exists in the schema. Roughly a day.
Stripe Checkout and webhook	half	The entitlement model is complete and tested; the five database columns exist and are write-protected. Needs a Stripe account, a checkout Edge Function, and a webhook writing those columns.
Photo sort screen Assign untagged captures to rooms	missing	Capture allows shooting untagged and job detail counts them, but there is no grid to sort them. Small — a grid plus room chips.
Product analytics PostHog, time-to- estimate	missing	The plan's headline metric is time from job creation to estimate sent. Nothing measures it yet.
Store submission	not started	Config is ready: bundle ids, permission strings, EAS profiles. Needs Apple and Google accounts. Review is slow — start it early.

What to do next, in order

- 01 Build it and hold it.** `eas build --profile development`, install on your phone, walk a room. Everything below is guesswork until this happens, and it will surface more than any amount of further code.
- 02 Build sign-in.** Nothing reaches a server without it, which means sync, the AI layer and the shareable link are all currently theoretical. This unblocks the most.
- 03 Point it at a real Supabase and watch the queues drain.** The offline spine is the most heavily tested part of the codebase and the least proven. One inspection in airplane mode, then signal.
- 04 Deploy the AI function and look at the answers.** `supabase functions deploy ai` with an Anthropic key. Then read the acceptance rate in settings and cut anything under 60%, as the plan says, rather than tuning it.
- 05 Go on a real inspection.** This was Week 1 Day 6 of the plan and it never happened. Six hours next to a contractor will change more than six days of building.
- 06 Then Stripe, the photo sort screen, and store submission.** In that order, and none of them before you have watched somebody use this.

The honest summary

The engineering is in good shape: the money arithmetic, the geometry, the offline queues and the tenant isolation are all covered by tests that assert real behaviour rather than shape. The risk has moved. It is no longer *“is the code right?”* — it is *“does a contractor want this, and does it hold up in a wet basement?”* Neither question gets answered by writing more of it.

Running it today

The dev server is up. It will **not** load in Expo Go — speech recognition and Sentry are native modules that need a development build:

```
# One time: a dev build for your phone (cloud, ~15 min)
npx eas login
npx eas build --profile development --platform android

# Then, each session
npx expo start --dev-client      # already running on 192.168.1.14:8081

# What works with no backend at all:
#   measure rooms, shoot photos, damage sheet, voice notes,
#   import prices, auto-scope, price an estimate, print a PDF
#
# What will not work until sign-in exists:
#   sync, the AI features, the shareable link
```

Everything else is verifiable right now without a phone: `npm test` (619 tests), `npm run typecheck`, `npm run db:verify` (applies all 9 migrations to a throwaway Postgres and runs 19 security assertions), and `npm run preview:docs` (writes a sample estimate and photo report you can open in a browser).